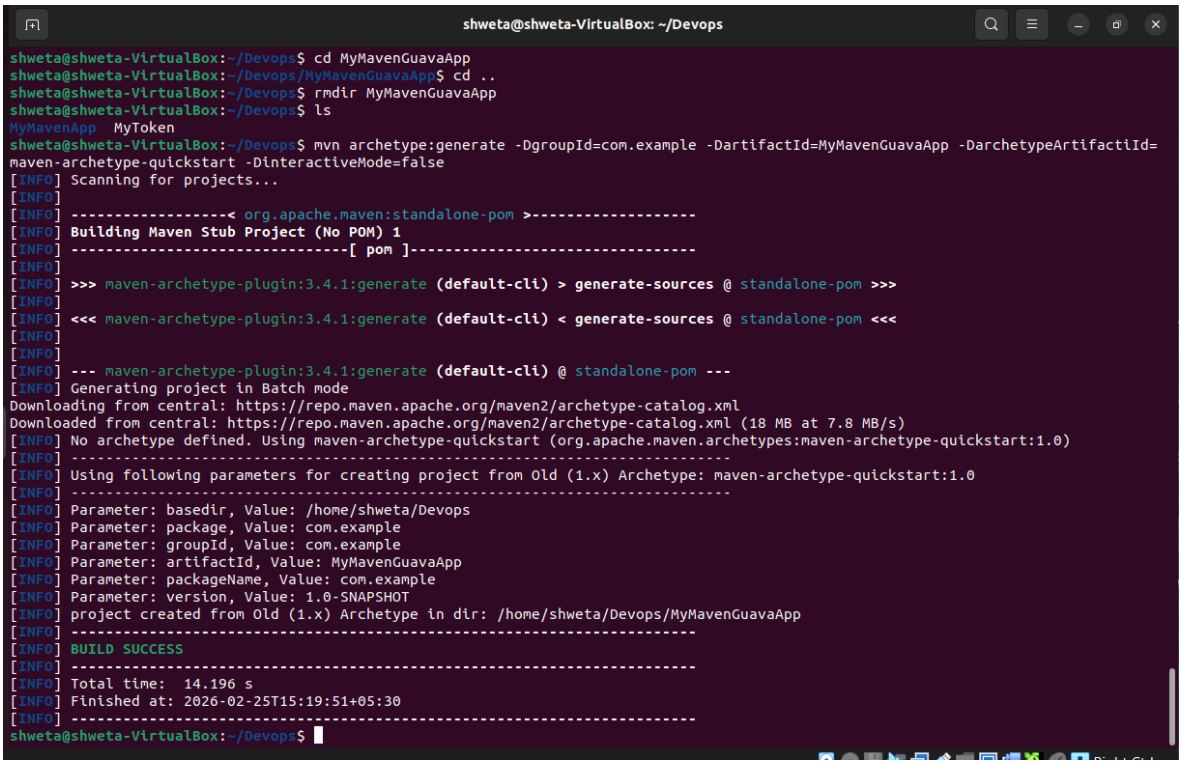
Program 2: Creating MyMavenGuavaApp Project

Step1: Open terminal -> cd DevOps -> Type below command to create MyMavenGuavaApp

mvn archetype:generate -DgroupId=com.example -

DartifactId=MyMavenGuavaApp - DarchetypeArtifactId=maven-

archetype-quickstart -DinteractiveMode=false



Step 2: cd MyMavenGuavaApp -> gedit pom.xml

This pom.xml file is created to configure the Maven project. It defines the project information, adds required dependencies like Guava and Commons IO libraries, and includes build plugins to compile the Java code, run unit tests, and package the application into an executable JAR file.

<project
xmlns="http://maven.apache.org/POM/4.0.0
"

```
xmlns:xsi="http://www.w3.org/2001/XMLS
chema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
  <artifactId>MyMavenGuavaApp</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenGuavaApp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
    <!-- https://mvnrepository.com/artifact/com.google.guava/guava -->
<dependency>
```

```
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>33.4.0-jre</version>
</dependency>
<!-- https://mvnrepository.com/artifact/commons-io/commons-io -->
<dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.18.0</version>
</dependency>
  </dependencies>
  <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
```

```
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.1.0</version>
        <configuration>
          <archive>
            <manifestEntries>
              <Main-Class>com.example.App</Main-Class>
```

```
            </manifestEntries>

          </archive>

        </configuration>

      </plugin>
</plugins>

</build>

</project>
```

Step 3: gedit App.java

This step opens the App.java file using the gedit editor. The code is pasted to create a Java program that uses the Google Guava library to create an immutable list and Apache Commons IO library to copy a file from source to destination.

```
package com.example;

// Import FileUtils class from Apache Commons IO library

// This is used to perform file operations like copying files import
org.apache.commons.io.FileUtils;

// Import File class to represent file and directory pathnames import java.io.File;

// Import IOException to handle input-output errors import java.io.IOException;

// Import ImmutableList from Google Guava library

// ImmutableList creates a list that cannot be modified after creation import
com.google.common.collect.ImmutableList;

// Define the main class public class App
{
```

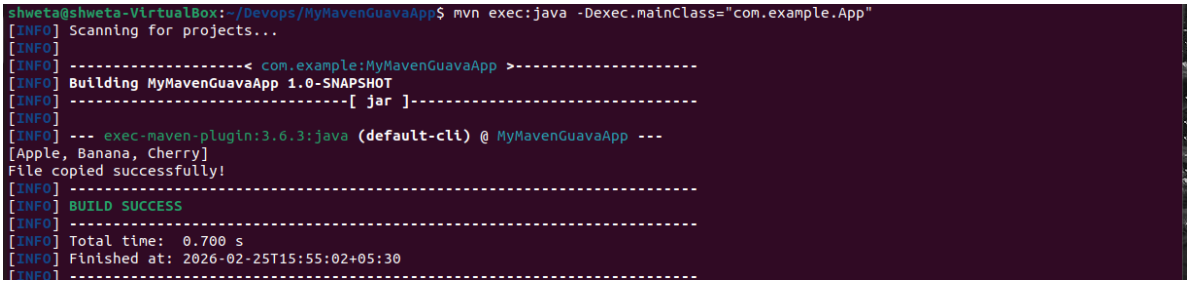
```
// Main method - program execution starts from here public static void main(
String[] args )
{

// Create an immutable list of strings (fruits)

// This list cannot be changed (no add/remove allowed)



ImmutableList<String> fruits = ImmutableList.of("Apple", "Banana", "Cherry");

// Print the list of fruits to the console System.out.println(fruits);

// Create a File object representing the source file File sourceFile = new
File("source.txt");

// Create a File object representing the destination file File destFile = new
File("destination.txt");

try {

// Copy the contents of source.txt to destination.txt

// Using Apache Commons IO FileUtils class FileUtils.copyFile(sourceFile,
destFile);

// Print success message if file is copied System.out.println("File copied
successfully!");
}

catch (IOException e) {

// If any error occurs during file copying,

// this block will handle the exception

// Print error message
```

```
System.err.println("Error occurred while copying file: " + e.getMessage());
}}}
```

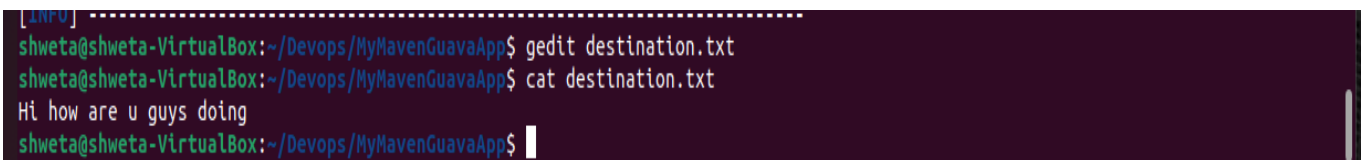
Step 4: mvn clean install

Step 5: Under the MyMavenGuavaApp project directory, create a file named source.txt and add some sample text such as **"Hello how are you guys doing"**. This file will act as the source file that the Java program copies to destination.txt during execution.

Step 6: mvn exec:java -Dexec.mainClass="com.example.App"



Step 7: After running the program, the destination.txt file will be automatically created inside the **MyMavenGuavaApp** directory. The command cat destination.txt is used to display the contents of the file and verify the output.



Pushing MyMavenGuavaApp to Github:

```
shweta@shweta-VirtualBox:~/Devops/MyMavenApp$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/shweta/Devops/MyMavenApp/.git/
shweta@shweta-VirtualBox:~/Devops/MyMavenApp$ git add .
shweta@shweta-VirtualBox:~/Devops/MyMavenApp$ git commit -m "initial commit"
[master (root-commit) 3356dc4] initial commit
 15 files changed, 197 insertions(+)
 create mode 100644 pom.xml
 create mode 100644 src/main/java/com/example/App.java
 create mode 100644 src/main/java/com/example/MainClass.java
 create mode 100644 src/test/java/com/example/AppTest.java
 create mode 100644 target/MyMavenApp-1.0-SNAPSHOT.jar
 create mode 100644 target/classes/com/example/App.class
 create mode 100644 target/classes/com/example/MainClass.class
 create mode 100644 target/maven-archiver/pom.properties
 create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
 create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
 create mode 100644 target/surefire-reports/TEST-com.example.AppTest.xml
 create mode 100644 target/surefire-reports/com.example.AppTest.txt
 create mode 100644 target/test-classes/com/example/AppTest.class
```

```
shweta@shweta-VirtualBox:~/Devops/MyMavenApp$ git push -u origin master
Enumerating objects: 41, done.
Counting objects: 100% (41/41), done.
Delta compression using up to 3 threads
Compressing objects: 100% (24/24), done.
Writing objects: 100% (41/41), 7.52 KiB | 1.50 MiB/s, done.
Total 41 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/Shweta311204/MyMavenApp.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
shweta@shweta-VirtualBox:~/Devops/MyMavenApp$
```

Setting Up Jenkins Pipeline for MyMavenGuavaApp

Step 1: To deploy **MyMavenGuavaApp**, open the terminal, cd into the **MyMavenGuavaApp** folder, create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub.

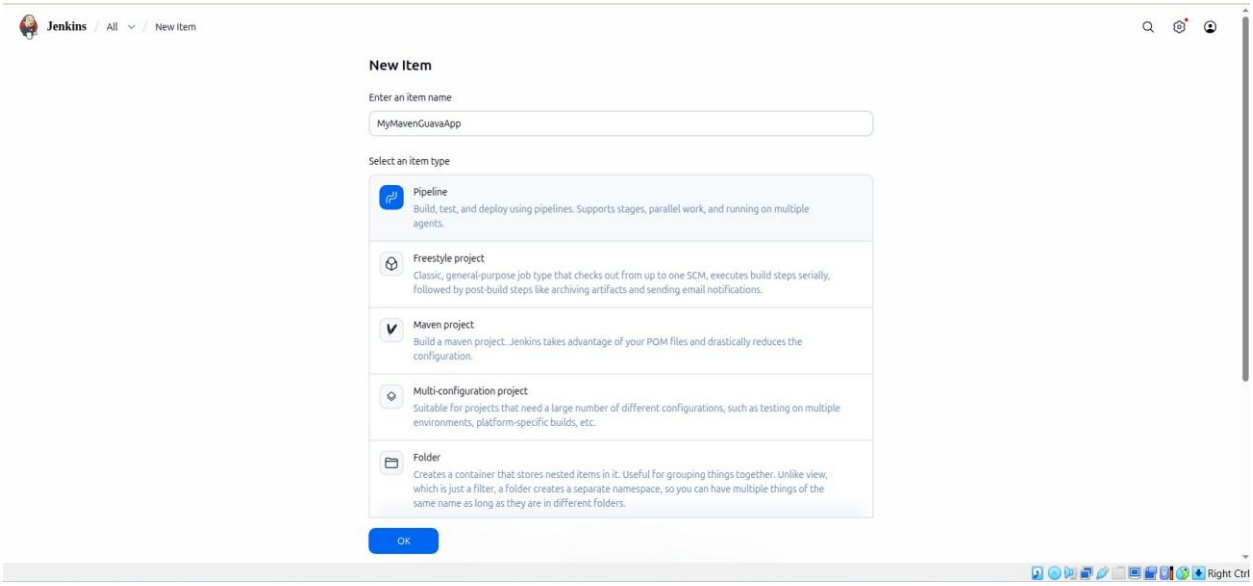
pipeline {

agent any // Use any available agent tools {

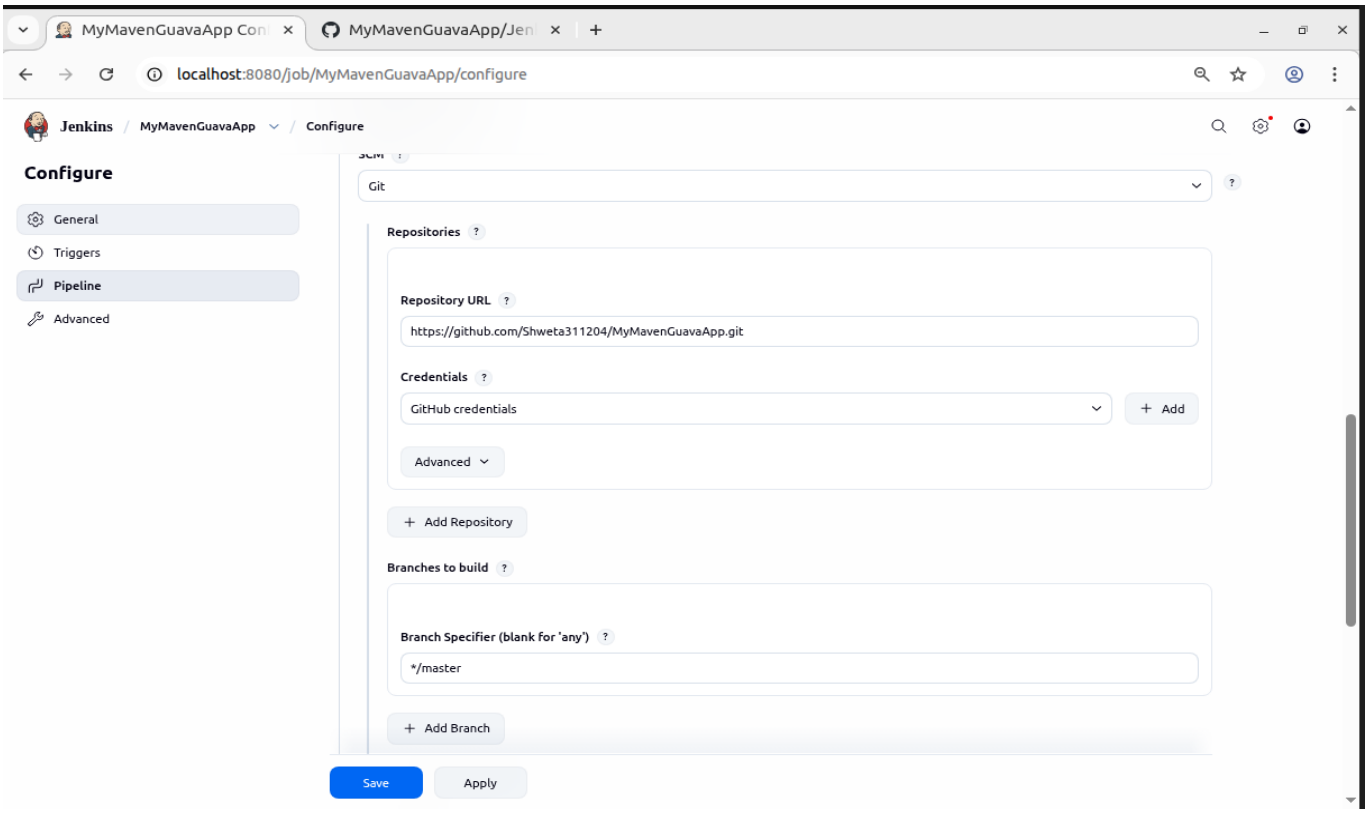
```
maven 'Maven'  // Ensure this matches the name configured in Jenkins


}

stages {

stage('Checkout') { steps {

git branch: 'master', url: 'https://github.com/Shweta311204/MyMavenGuavaApp.git'
}
}

stage('Build') { steps {

sh 'mvn clean package'  // Run Maven build
}
}

stage('Test') { steps {

sh 'mvn test'  // Run unit tests
}
}

stage('Run Application') { steps {

// Start the JAR application

sh 'java -jar target/MyMavenGuavaApp-1.0-SNAPSHOT.jar'
}
}
}

post {

success {

echo 'Build and deployment successful!'
}
```

```
failure {

echo 'Build failed!'
}
}
}
```

Step 2: To deploy **MyMavenGuavaApp**, you first need to create a Jenkins pipeline job that will use the Jenkinsfile from your project repository. Then, go to the Jenkins home page and click on **"New Item"** to create a job, enter the item name (e.g., *MyMavenGuavaApp*), and select **Pipeline** as the project type. This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click **"OK"** to proceed to the job configuration page.



Step 3: In **MyMavenGuavaApp**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of **MyMavenGuavaApp**. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the MyMavenGuavaApp folder and pushed to GitHub. Then click **Apply** and **Save** to finalize the configuration.



Step 4: After clicking **Apply and Save**, go back to **MyGradlePipelineApp** in Jenkins and click **Build Now**. If you get an error like:

java.lang.NoClassDefFoundError: com/google/common/collect/ImmutableList

this error occurs when the application is unable to find required external libraries (such as **Google Guava**) at runtime. Even though the project builds successfully, the dependencies are not included inside the final JAR file, so Jenkins fails while executing the program.

To solve this issue, we use the **Maven Shade Plugin**, which creates a **fat (uber) JAR** by bundling all project dependencies along with the application code into a single executable JAR file. This ensures that no external libraries are missing during execution.

Replace the existing plugin with the following:

<plugin>

<groupId>org.apache.maven.plugins</groupId>

```
<artifactId>maven-shade-plugin</artifactId>

<version>3.2.4</version>

<executions>

<execution>

<phase>package</phase>

<goals>

<goal>shade</goal>

</goals>

<configuration>

<transformers>

<transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTrans
former">

<mainClass>com.example.App</mainClass>

</transformer>

</transformers>

</configuration>

</execution>

</executions>

</plugin>
```

Step 5: After making the changes, go back to **MyGradlePipelineApp** in Jenkins and click **Build Now**. The build should now complete successfully without any errors.

This happens because the **Maven Shade Plugin** creates a **fat (uber) JAR**, which includes all required dependencies (like Guava, Selenium, etc.) inside the final build. So, during execution, Jenkins is able to find all the necessary classes, and issues like java.lang.NoClassDefFoundError are resolved.